



INNOPOLIS UNIVERSITY

System and Network Administration

Docker

HomeWork 8

**Sarmat Lutfullin
B23-CBS-02**

1 Questions to answer

1. Compare and contrast ENTRYPOINT and CMD in Dockerfile. In what situation would you use each of them?

Purpose:

- ENTRYPOINT
 - Defines the main command that runs inside the container, making the container behave like an executable.
- CMD
 - Provides default arguments for the container, which can be overridden at runtime.

Overridability:

- ENTRYPOINT
 - Only with **docker run --entrypoint**.
- CMD
 - Can be overridden by passing a command in **docker run**.

Use cases:

- ENTRYPOINT
 - We use **ENTRYPOINT** when we want to define a container's primary purpose and we want the container to behave like an executable. For example, if we have a container that is meant to run a specific application, we would use ENTRYPOINT to specify that application.
- CMD
 - We use **CMD** when we want to provide default arguments or a default command that can be easily overridden by the user. For example, if we have a container that can run different commands based on user input, we would use CMD to specify the default command.

2. List five security precautions you will take when building or deploying a Docker resource (image or container).

Use minimal base images:

Smaller base images reduce the attack surface by including only the necessary dependencies and libraries.

Avoid running containers as root:

Running containers as the root user increases the risk of privilege escalation attacks. We can use such tools as Trivy, Docker Scout and Clair.

Scan images for vulnerabilities:

Docker images may contain vulnerabilities in their dependencies or libraries.

Limit container privileges and capabilities:

Containers with excessive privileges can compromise the host system. Use the least privilege principle by restricting container capabilities. Run containers in read-only mode to prevent modifications to the filesystem.

Implement network security:

We can use firewalls, network policies and docker's built-in networking features to isolate containers and restrict communication.

3. Show a single line command that will remove all exited Docker containers. Do not use any text filtering editor. Show test results.

```
sarmat@isarmatt:~$ sudo docker ps -a
[sudo] password for sarmat:
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
PORTS         NAMES
9ae3130ff0d8   hello-world "/hello"               10 minutes ago Exited (0) 10 minut
es ago
a3a9883a1e76   alpine     "echo Hello"           11 minutes ago Exited (0) 11 minut
es ago
sarmat@isarmatt:~$ sudo docker rm $(docker ps -aq -f status=exited)
9ae3130ff0d8
a3a9883a1e76
sarmat@isarmatt:~$ sudo docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS         NAMES
sarmat@isarmatt:~$
```

To remove exited containers, we can use next command **docker rm \$(docker ps -aq -f status=exited)**.

- `docker ps -aq -f status=exited` - Lists only exited container IDs.
- `docker rm $(...)` - Removes all listed container IDs.

4. Show how you can copy files to a running container without entering the container's interactive shell

To copy files to a running container we can use the following command:

docker cp <local_file_or_directory><container_id_or_name>:<container_path>

```
sarmat@isarmatt:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
e6da0f5c0701   alpine    "tail -f /dev/null"     6 seconds ago Up 6 seconds
14609aac537d   alpine    "tail -f /dev/null"     7 minutes ago Exited (137) 4 mi
nutes ago
my_container1
my_container
```

```
sarmat@isarmatt:~$ echo "Hello, world!" > example.txt
```

```
sarmat@isarmatt:~$ docker cp example.txt my_container1:/example.txt
Successfully copied 2.05kB to my_container1:/example.txt
sarmat@isarmatt:~$ docker exec my_container1 ls -l /example.txt
-rw-rw-r-- 1 1000 1000 14 Mar 17 10:57 /example.txt
sarmat@isarmatt:~$ docker exec my_container1 cat /example.txt
Hello, world!
```

5. Create a dockerized web application running on nginx. The web index page index.html should be located on your host machine. The directory containing the index page should be mounted to the container and served from there. This means that you should be able to modify the web index page on your host machine without interacting with the container. Show all steps taken for the configuration including the test results.

```
sarmat@isarmatt:~$ mkdir ~/my-web-app
sarmat@isarmatt:~$ echo "<h1>My web app.</h1>" > ~/my-web-app/index.html
sarmat@isarmatt:~$ touch ~/my-web-app/Dockerfile
sarmat@isarmatt:~$ cd my-web-app && open Dockerfile
```

Figure 1: index.html and Dockerfile creation

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Figure 2: Dockerfile settings

```
events {
    worker_connections 1024;
}

http {
    server {
        listen 80;
        root /usr/share/nginx/html;
        index index.html;

        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

Figure 3: nginx configuration

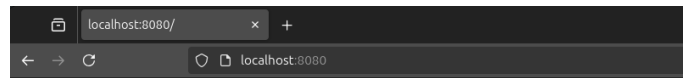
```
sarnat@1sarnatt:~/my-web-app$ docker build -t my-nginx-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 3.072kB
Step 1/4 : FROM nginx:alpine
alpine: Pulling from library/nginx
6e771e15690e: Already exists
0be31969a6d1: Pull complete
9e170776f94c: Pull complete
83f1386059fa: Pull complete
8ea77ffafa6e: Pull complete
76f8ad18306e: Pull complete
3c5c25f3816a: Pull complete
0a329c61f9e1: Pull complete
```

Figure 4: building docker image

```
sarnat@1sarnatt:~/my-web-app$ docker run -d --name my-web-container -p 8080:80 -v /my-web-app:/usr/share/nginx/html my-nginx-app
6c82432e5d10183eca54d20b83b1972950386558b9d4201af3c5c295323d8199
```

Figure 5: running the container



My web app.

Figure 6: testing in browser

```
sarnat@isarnatt:~/my-web-app$ ls
Dockerfile  index.html  nginx.conf
sarnat@isarnatt:~/my-web-app$ echo "<h1>Welcome to My Web App</h1>"> index.html
sarnat@isarnatt:~/my-web-app$
```

Figure 7: modifying index on host machine

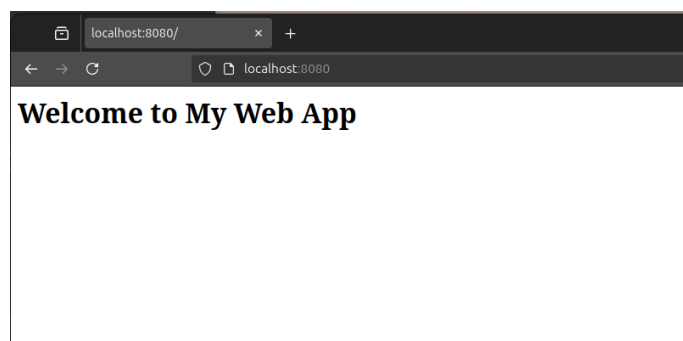


Figure 8: refresh the browser

6. Setup rsyslog on your host machine as a central logging server. Create a Docker container and configure it to forward its log to your central logging server.

```
sarmat@isarmatt:~$ sudo apt update
sudo apt install rsyslog
[sudo] password for sarmat:
Hit:1 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Get:2 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease [126 kB]
Get:3 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease [126 kB]
Get:4 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease [126 kB]
Get:5 http://ports.ubuntu.com/ubuntu-ports noble/main Icons (48x48) [106 kB]
Get:6 http://ports.ubuntu.com/ubuntu-ports noble/main Icons (64x64) [156 kB]
Get:7 http://ports.ubuntu.com/ubuntu-ports noble/main Icons (64x64@2) [21.8
Get:8 http://ports.ubuntu.com/ubuntu-ports noble/universe Icons (48x48) [3.7
```

Figure 9: Install rsyslog

```
module(load="imuxsock") # provides support for local system logging
#module(load="immark") # provides --MARK-- message capability

# provides UDP syslog reception
module(load="imudp")
input(type="imudp" port="514")

# provides TCP syslog reception
module(load="imtcp")
input(type="imtcp" port="514")
```

Figure 10: Configure rsyslog as central logging server

```
sarmat@isarmatt:~$ sudo systemctl enable rsyslog
sarmat@isarmatt:~$ sudo nano /etc/rsyslog.conf
sarmat@isarmatt:~$ sudo systemctl restart rsyslog
sarmat@isarmatt:~$ sudo ufw allow 514/udp
sudo ufw allow 514/tcp
sudo ufw reload
Skipping adding existing rule
Skipping adding existing rule (v6)
Skipping adding existing rule
Skipping adding existing rule (v6)
Firewall not enabled (skipping reload)
```

Figure 11: Allow traffic on port 514

```
GNU nano 7.2 remote.conf
$template RemoteLogs, "/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
*. * ?RemoteLogs
& stop
```

Figure 12: To organize logs per client, we can edit the rsyslog configuration

```

sarnat@isarnatt:~$ docker run -dit --name log_client \
--log-driver=syslog \
--log-opt syslog-address=tcp://192.168.64.16:514 \
--log-opt tag="docker_log_client" \
ubuntu bash
55e79f757447267d22caa33789934b09c39e41fcea0e5210fc0de663af71205d
sarnat@isarnatt:~$ docker exec -it log_client bash -c 'echo "Test log from Docker" | logger'
sarnat@isarnatt:~$ sudo grep "Test log from Docker" /var/log/syslog
sarnat@isarnatt:~$ ls -lah /var/log/remote/
total 12K
drwxr-xr-x  3 syslog syslog 4.0K Mar 17 18:00 .
drwxrwxr-x 18 root    syslog 4.0K Mar 17 18:07 ..
drwxr-xr-x  2 syslog syslog 4.0K Mar 17 18:05 isarnatt

```

Figure 13: Running docker and verifying the logs are being sent

2 Bonus Question

1. Dockerize any open source application of your choice, and host it on Docker hub. Share link to the repository.

```
sarnat@isarnatt:~$ mkdir nginx-dockerized
cd nginx-dockerized
sarnat@isarnatt:~/nginx-dockerized$ nano index.html
```

Figure 14: preparing environment

```
GNU nano 7.2 index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Dockerized Nginx</title>
</head>
<body>
  <h1>Welcome to my Nginx running inside Docker.</h1>
</body>
</html>
```

Figure 15: creating index.html

```
sarnat@isarnatt:~/nginx-dockerized$ nano Dockerfile
sarnat@isarnatt:~/nginx-dockerized$ docker build -t my-nginx .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 3.072kB
Step 1/4 : FROM nginx:latest
--> 678546cdd20c
Step 2/4 : COPY index.html /usr/share/nginx/html/index.html
--> 4393ea8d2997
Step 3/4 : EXPOSE 80
--> Running in 83849bdf0535
--> Removed intermediate container 83849bdf0535
--> 2bef23935855
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
--> Running in d19a17642564
--> Removed intermediate container d19a17642564
--> c2d7820364f7
Successfully built c2d7820364f7
Successfully tagged my-nginx:latest
```

Figure 16: creating dockerfile

```
GNU nano 7.2 Dockerfile
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Figure 17: changing dockerfile

```
sarmat@1sarmatt:~/nginx-dockerized$ docker run -d -p 8080:80 my-nginx
1dcc8829a826c1917be6699f809679fc27abeebd900de4460781ae2359682cc5
sarmat@1sarmatt:~/nginx-dockerized$ docker login
Log in with your Docker ID or email address to push and pull images from Docker
Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to crea
te one.
You can log in with your password or a Personal Access Token (PAT). Using a limi
ted-scope PAT grants better security and is required for organizations using SSO
. Learn more at https://docs.docker.com/go/access-tokens/

Username: 1sarmatt
Password:
WARNING! Your password will be stored unencrypted in /home/sarmat/.docker/config
.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

Figure 18: running docker



Welcome to my Nginx running inside Docker.

Figure 19: open locally

```
sarmat@1sarmatt:~/nginx-dockerized$ docker tag my-nginx 1sarmatt/nginx-dockerize
d:latest
docker push 1sarmatt/nginx-dockerized:latest
The push refers to repository [docker.io/1sarmatt/nginx-dockerized]
f451f52a36ff: Pushed
e22d4b0b4d6e: Mounted from library/nginx
14916ed5c789: Mounted from library/nginx
6280cdec38c1: Mounted from library/nginx
de84c2fb18e8: Mounted from library/nginx
55c80bdf194c: Mounted from library/nginx
a0e17087bc6a: Mounted from library/nginx
52d51720ba2d: Mounted from library/nginx
latest: digest: sha256:c92975e203d7706a1507919f88372af9692ebdc50a4eb63c5496082f8
74be1bd size: 1986
```

Figure 20: push to docker hub: <https://hub.docker.com/repositories/1sarmatt>